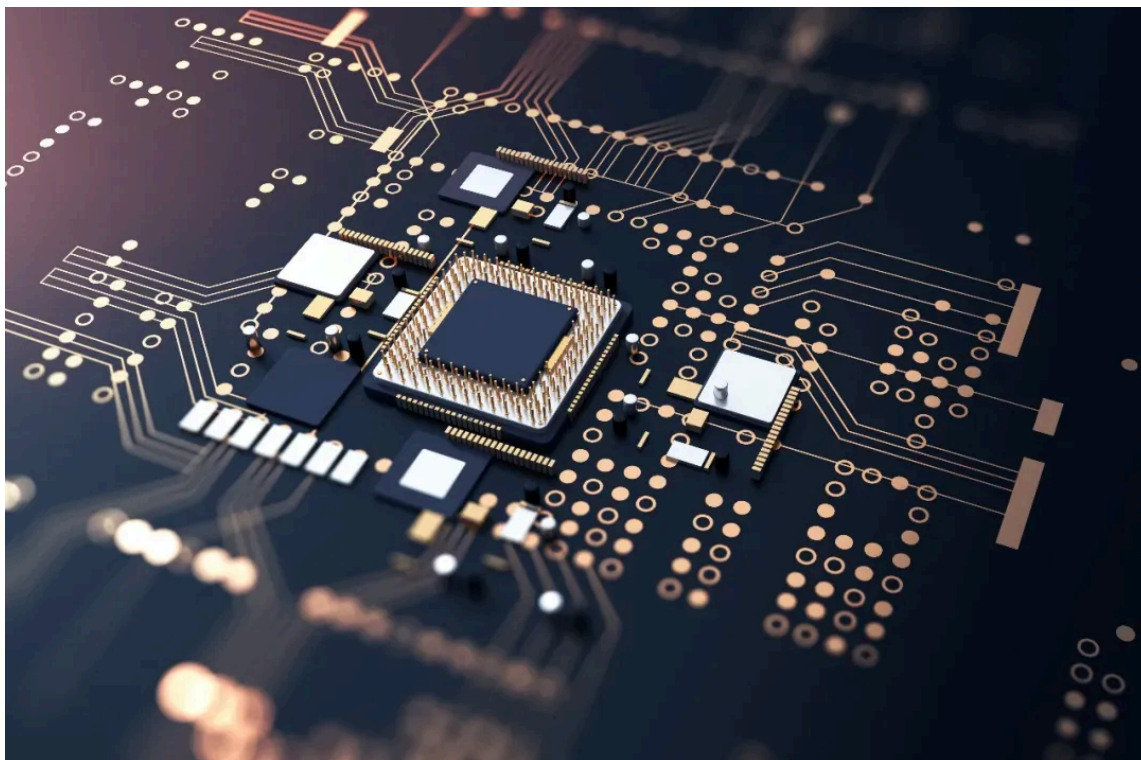


Agent核心技术概念与范式发生了哪些演变以及背后的思考

飞樯 阿里技术 2026年6月1日 18:18 浙江



这是2026年的第19篇文章

(本文阅读时间: 约20分钟)

前言

近几年，随着基模能力的快速升级与迭代，Agent 领域迎来了爆发式的增长。特别是近期，像 Claude Code、Codex、OpenClaw、Hermes 等可以说是全新一代的 Agent 产品和框架在不断涌现，Agent 的能力相比早期版本也出现了进一步的跃升，进一步推动了整个 Agent 生态的繁荣发展。

眼下，我们正处在 Agent 发展的快速变化期，现在的许多主流 Agent 技术在架构理念、实现路径上，与早期的 Agent 技术已经发生了较大的变化。然而，有许多的同学在学习和阅读 Agent 相关的文章内容的时候，仍然学习到了比较早期的内容，导致这里面有许多新旧技术概念混在一起，很容易搞不清楚，加上部分内容学习来源现阶段还在转载比较早期的文章，导致大家越了解越迷糊。我之所以突然产生写这篇文章的想法，就是因为有不少同学在我交流时提到的一些技术概念，偶尔会让我产生一丝“穿越感”。

Agent 相关的技术发展并非一蹴而就，很多技术概念前后之间也是有一定的相关、继承关系的。即使发生了演变前后的技术，也并非简单的替代关系，甚至还可以相互结合使用。因此，在 Agent 的技术理念已经发生较大的变化的时候，我觉得非常有必要对 Agent 演化前后的范式进行一次更深入对比，梳理出“Agent 技术是怎样变化的？为什么会这样变化？”这一问题的逻辑。

如果搞不清楚 Agent 技术范式背后的演化逻辑，很容易陷入“为了升级新架构而升级架构”或者“盲目追求最新技术概念”的误区。因此，本文旨在结合最新的行业实践和技术趋势，以及个人对 Agent 的长期理解，详细拆解 Agent 的演化范式，希望能帮助大家在纷繁复杂的技术浪潮中，理清思路，找到最适合特定场景的技术选型。

01

从被动响应到自进化：Agent 发展的四个阶段

回顾 2023~2026 这三年的时间，Agent 的技术形态并非线性平滑过渡，而是经历了四个我认为比较有显著特征的“四个阶段”。理解这四个阶段的演进过程，有助于我们看清当前技术选型的底层脉络。

阶段一：早期 Agent（被动式 ReAct）

2023年是LLM爆发的元年，也可以说是 Agent 概念的**启蒙期**。这一阶段的代表性理论源自 Lilian Weng 的那篇著名博客《LLM Powered Autonomous Agents》，它定义了基于大模型的 Agent 基本架构：**LLM + Planning + Tools + Memory**，给出了当时早期 Agent 比较理想的模型。这个时期，也有如AgentGPT、AutoGen、MetaGPT等等各种开源项目实现。

这个阶段的 Agent 本质上是“**被动式响应**”的。Agent 的核心架构基本上是基于比较初步的 **ReAct 架构 (Reasoning + Acting)**，基本上是符合单步的“Reasoning → Observe → Response”这样的过程链条，受限于基础模型的效果，能够做好3轮以上的 Reasoning 的模型都不多。

早期阶段的 Agent 基本上有如下几个特点：

- 交互形态**：类似于增强版的 Chatbot，处于一种“一问一答”或“指令-执行”的聊天状态。
- 能力边界**：严重依赖用户的明确指令。虽然引入了**思维链 (CoT, Chain of Thought)** 和简单的**工具调用 (Function Call)** 链条，但相对来将，基本上只能完成单点、短链路的小任务。
- 局限性**：缺乏长期规划能力，一旦任务复杂度超出上下文窗口或逻辑链条过长，极易出现偏离或中断。

阶段二：工作流 Agent (结构化与可控性)

在2024年时期，随着 to B 业务对稳定性要求的提升，纯靠 ReAct 这种“理想方式”解决不了复杂问题的情况下，**Agentic Workflow** 成为了主流。这一阶段核心理念是：**用工程化的约束来弥补模型的不确定性**。像 LangGraph、Dify 等都提供 Workflow 的流程编排。

与早期 Agent 阶段的纯模型驱动不同，Workflow Agent 引入了大量的硬约束和流程编排，我觉得这也可以理解为早期的 Harness（驾驭工程里的“约束”）吧，虽然当时没有这个概念，但所做事情的目标本质是一样的。这个阶段的 Agent 主要是如下几个特点：

- 架构特征**：要么是整个大框架是一个固定的 Workflow，**关键节点嵌入 LLM**；要么是 LLM 作为中枢，调用预定义好的子 **Workflow**。是一套比较重的 Harness，虽然牺牲了一定的灵活性，但换来了极高的可控性和可解释性。
- 应用场景**：Workflow 在 to B 领域极受欢迎。因为很多**企业服务**或**日常重复性工作**，并不需要真正的“智能决策”，只需要按照步骤 1、2、3 按时、按量、保质地完成即可。
- 价值体现**：对于**非长尾、非极度复杂**的场景，Workflow Agent 依然是目前性价比最高、落地最稳定的方案。时至今日，仍有大量企业在使用这种形态，因为它能确保效果的下限。

阶段三：自主 Agent (复杂规划与长程任务)

个人认为，2025年是 Agent 迈向“**自主性**”的**关键转折点**。先是以 Manus 为代表的通用 Agent 的火爆，以及 Claude Code、Codex 等 AI Coding Agent 的出现等等，标志着 Agent 能力再一次质的飞跃。随后在2026年初火爆的 OpenClaw 等框架，继续扩大了受众群体，进一步巩固了这一技术趋势。

这一阶段的 Agent 可以被称为“自主 Agent” (Autonomous Agent)，主要特征如下：

- 核心变化**：它不再满足于快速调用几个工具后给出结论，而是具备了复杂的 **Planning（规划）** 能力。面对用户模糊或宏大的需求，它能自行拆解任务、规划路径、调用工具，并进行多轮迭代。
- 长程任务能力**：只要用户清晰描述需求，并设定好**开发规范（Specs）**，Agent 就可以连续运行很长时间，自主处理企业级的项目代码或复杂业务流程。
- 自我校验**：配合轻量级的 **Harness** 或**自我校验机制**，模型能够在长程运行中不断修正错误，最终交付高质量的结果。这是从“辅助者”向“执行者”角色的根本转变。

阶段四：自进化 Agent（持续学习与自我升级）

随着2026年 Hermes Agent 等新一代框架的兴起，再配合上 LLM-Wiki 等这些开源项目，Agent 可以自我沉淀Skill、自我沉淀知识库，甚至可以通过 RL 训练来提升模型能力，让 Agent 的发展进入了“**自进化（Self-Evolving）**”的新阶段。

这一阶段的核心本质，是开始解决“静态模型”与“动态世界”之间的矛盾。

- 机制原理**：Agent 不仅仅是在完成任务，更是在完成任务的过程中**沉淀经验**。通过记忆模块、反馈循环和自我反思机制，Agent 能够将从前一次任务中获得的教训转化为新的知识或策略。
- 最终目标**：实现“**越用越好用**”。Agent 能够根据历史交互数据，自动优化自身的提示词、工具选择策略甚至微调局部模型参数，实现自我升级和进化。
- 意义**：这标志着 Agent 从“一次性消耗品”变成了“**可积累资产**”，为构建真正具备长期生命力的数字员工奠定了基础。

从最早期的 ReAct Agent，到结构化 Workflow Agent，再到后面可以自主规划长程任务的 Agent，直至2026年开始出现的自进化 Agent，Agent 的范式演化清晰地展示了一条从“简单交互”到“复杂执行”，再到“智能成长”的技术进阶之路。需要注意的是，这四个阶段并非完全的替代关系，而是**并存且互补**的。在实际落地中，我们需要根据业务的复杂度、对稳定性的要求以及成本预算，选择合适的 Agent 范式，或者将多种范式组合使用。接下来，我们将从更深入的技术概念的角度，来展开介绍核心技术的前后演进变化。

02

六个核心 Agent 技术概念前后变化的思考

现如今，创建一个哪怕是最轻量级的 Agent，除了最关键的 Agent Loop，还会涉及到 Prompt、Planning、Memory、Tools、Workflow、Environment等各个方面，今天我就以这六个最核心的技术维度，展开来介绍这些概念和实现都**发生了哪些变化，以及为什么会发生这些变化？**

Prompt：深耦合 → 渐进式加载

既然要深入聊一下 Agent 里面各项技术的核心细节变化，那么就从大家最早开始接触LLM，也是写的最多、最熟悉的**提示词 (Prompt)** 开始吧。

回想早期构建 Agent 的阶段，我们绝大部分的精力都耗费在撰写 Prompt 上。当时为了解决特定领域或独立场景的问题，我们的做法往往是“**一个任务创建一个Agent**”。比如，为了完成一篇高质量的文章，我们会拆解出多个 Agent：一个负责撰写初稿的“写作 Agent”，一个负责润色优化的“编辑 Agent”，还有一个负责生成配图的“绘图 Agent”等等。每个 Agent 的背后，都对应着一段精心调试、独立存在的 System Prompt，这个 Prompt 里需要包括 Agent 的人设、任务目标、任务要求、约束条件、注意事项、各种示例等等。这种模式下，Prompt Engineering 几乎等同于针对每个任务单独写一段“小作文”，不仅维护成本极高，而且随着场景增多，Prompt 的管理变得极其混乱。

但随着实践的深入，我们发现这种将“**系统级指令**”与“**任务要求&细节**”紧耦合的方式存在明显的瓶颈。于是，在 Agent 近期的技术演进中，出现了许多 System Prompt 层面的“**解耦策略**”。核心思路是：**尽量固化 System Prompt，将动态的、具体的任务要求剥离出来**。所以，现在的 Agent 系统的 System Prompt，基本上只保留最底层、最通用的系统级指令和基本行为规范，所需要的一些工具信息、Skill 使用方式，基本保留的都是比较“固定”的部分，不太容易变化，使其保持极度的“稳定”。而原本堆积在 System Prompt 中的大量具体的、细节的，比如**任务要求、领域知识、人设规范等“动态内容”**，则被拆解并存储到了外部的文件系统中，然后通过**渐进式披露 (Progressive Disclosure)** 的方式来进行读取和加载。

具体来说，Prompt 的动态性主要分为两个方向：

- **Skill 层面的沉淀**：我们将执行某项具体任务的方法论、步骤要求、领域约束等，沉淀为独立的 Markdown 文件，比如 SKILL.md 等。这些文件构成了 Agent 的“技能库”或者“要求库”，Agent 在执行特定任务时，会动态的渐进式披露加载对应的 Skill 中的 Markdown 文件，从而获取具体的操作指南。

- **配置文件存储**：对于人设定义、用户偏好、搜索规则等通用规范，我们将其存储在类似 USER.md、SOUL.md 或 CLAUDE.md、AGENTS.md 这样的配置文件中。同样通过渐进式

加载文件系统的方式，实现了对Prompt内容的模块化管理。

这种从“单体大System Prompt”到“System Prompt + 渐进式加载上下文文件”的转变，主要是“**上下文的组织形式发生了变化**”，这样做让 System Prompt 变得更加纯粹和稳定，而将易变的业务逻辑和领域知识通过结构化的 Markdown 文件进行灵活挂载。这不仅降低了维护复杂度，也让Agent在面对不同场景时，能够更灵活地组合所需的上下文信息，实现了真正的“动静分离”。

Planning：思维链 → 复杂长程任务

Agent 演化过程中，第二个显著的变化发生在 Agent 的“**规划（Planning）**”层面。

还是回到早期 Agent 的理论基础，在 Lilian Weng 的《LLM Powered Autonomous Agents》一文中的 Planning 在刚提出的时候是一个非常新鲜的概念，这让很多人都坚信：一个**真正会对任务做“Planning”的 Agent 才是真正意义上的 Agent**。然而，在那个基础模型还不够强的时代，Planning 的实现还是相对比较朴素的，主要就是依赖大模型原生的思维链（CoT，Chain of Thought）能力，比如通过类似“**Let's think step by step**”这样的提示词引导模型进行**线性的、串行的逻辑推导**。这种模式在处理简单任务时尚可应付，但在面对复杂场景时，往往显得力不从心，容易陷入逻辑断层或死循环。

然而，随着基础模型推理能力的飞速迭代，尤其是在 Reasoning 能力的显著增强的今天，如今的 Planning 机制其实已经发生了质的飞跃。现在的 Agent 不再仅仅满足于单步的思考，而是具备了更高级的如下的能力：

1.复杂问题的结构化分解：Agent 能够主动将一个宏大的、模糊的目标拆解为多个可执行的子任务（Sub-tasks），并生成结构化的 Todo List。

2.多步协同与长程推理：基于生成的任务列表，Agent 能够按步骤有序执行，并在执行过程中动态调整计划。这种能力使得 Agent 能够处理具有极长上下文依赖的复杂任务，保持逻辑的一致性和连贯性。

3.子 Agent 的动态构建：在更先进的架构中，Planning 甚至涉及到根据子任务的需求，动态实例化或调用特定的子 Agent 来专项解决某个环节的问题，实现了从“单体思考”到“协同作战”的转变。

Planning 层面能够做到这样的演化的核心驱动力，归根结底在于“**底层基座模型推理能力升级**”所带来的。随着模型在逻辑推理、长文本理解以及复杂指令遵循上的表现越来越强，Agent 的 Planning 模块也从简单的“提示词技巧”演变成了真正的“智能决策中枢”，能够胜任更加复杂、长周期的自主任务规划。

Memory：检索增强 → 文件系统化

第三个关键的变化，我们来看下“记忆（Memory）”模块的演进上。在 Lilian Weng 早期的经典架构中，Memory 被划分为短期记忆（Short-term Memory）和长期记忆（Long-term Memory）。当时的定义相对直观：

- 短期记忆：**主要指对话上下文，包括 System Prompt、历史对话中的 User 和 Assistant 回复等。
- 长期记忆：**主要指外部知识库，通常通过 RAG 从向量数据库中检索相关的文档或知识片段，作为背景信息注入给大模型。

然而，随着 Agent 应用场景的复杂化，这种简单的 Memory 管理方式已无法满足需求，两个维度的记忆机制都发生了演变。

短期记忆层面（Short-term Memory），核心挑战从“存储”转向了“管理”与“压缩”。由于 Context Window（上下文窗口）有限且成本敏感，为了保证长上下文下 Agent 的效果，以及高效利用有限的 token 就成为关键。从 OpenClaw、Hermes 这些最佳实践来看，上下文不再只是简单堆砌历史对话，而是引入了多种**记忆压缩策略**：

- 阈值控制：**基于固定 token 数或动态语义密度阈值触发压缩。

- 结构化摘要**：对中间过程的对话进行 Summary 提炼，同时保留头尾的关键指令和最终结论，确保核心意图不丢失。

- 重点提取**：从冗长的对话流中提取关键事实或状态变化，剔除无关噪音。这些手段使得短期记忆更加精炼、高密度，显著提升了模型在长对话中的注意力集中度。

长期记忆层面 (Long-term Memory)，变化相对更大，逐步在从“向量数据库主导”向“文件系统主导”回归的趋势，并细分为两个子方向：

- 事项型记忆 (Episodic Memory)**：针对用户偏好、历史行为、每日待办等动态变化的“事实”，越来越多的框架，比如 OpenClaw、Hermes Agent等就倾向于使用文件系统进行记录。例如，通过生成 MEMORY.md 或每日的Memory日志文件，以结构化的 Markdown 格式存储关键事件。这种方式比向量检索更可控、更易读，也便于 Agent 直接读取和理解时间序列上的状态变化。

- 知识型记忆 (Semantic Memory)**：随着 Karpathy 等提出的 LLM-Wiki、GBrain这类本地化知识库理念的普及，大模型在知识存储上也在发生变化。传统的纯 RAG 方案正在被更灵活的**本地文件系统 + Obsidian 等笔记工具**所补充甚至替代。通过这些文件系统知识工具，Agent 可以直接访问组织良好的 Markdown 知识库，非常适合个人知识库的构建使用。当然，如果是企业级的知识库，存储了海量知识的背景下，仅通过这类“文件系统即记忆”的模式，还是不太够的，只通过

grep 类的命令关键词检索，很容易不准确。因此，除了文件系统之外，还需要搭配 QMD 或者 SQLite 等轻量化的向量化检索机制，甚至更加高度复杂的场景还需要企业级的向量检索，才能不仅保留 RAG 的海量知识优势，还赋予了开发者通过目录结构、标签、链接来显式组织知识的能力，使得知识的召回更加精准和可解释。

综上所述，Memory 的演进本质上是开始**从纯向量文本检索走向“文件系统化的沉淀+向量检索混合管理”**。无论是短期的对话压缩，还是长期的事项记录与知识沉淀，都在追求更高的记忆效果、可读性和效率的均衡。

Tools: Function Call → CLI / Script

第四个部分，也是我认为 Agent范式中变化最为大的一个部分，在于“工具 (Tools)”执行方式的变化。

回顾 Agent 发展的早期阶段，工具调用的主流范式是 **Function Call**。我们需要针对具体的业务场景，将系统能力封装成标准的 API，并注册为模型可调用的函数。这种方式虽然实现了模型与外部系统的连接，但存在一个显著的痛点：**极高的开发与维护成本**。现实中，大量的系统或数据源并没有现成的 API 可供调用，为了弥补这一缺口，团队往往需要投入大量精力去“补全”API，这不仅费时费力，而且随着工具数量的膨胀，API Schema 的管理变得极其复杂。随后出现的 **MCP (Model Context Protocol)** 虽然在协议层面优化了工具的注册与发现机制，实现了“一次注册，自动暴露”，但这本质上仍停留在接口标准化的层面，并未从根本上改变工具调用的底层逻辑。

而真正的范式转移发生在两个关键维度的变化：**CLI 命令行原生化与Script 脚本化**。

首先，**CLI (命令行界面)**，一种除了程序员之外，大部分人比较陌生的工具，在 Agent 时代再次焕新了它的生机。其实 CLI 这种枯燥的命令行和参数，对人类来讲并不是一种友好的交互方式，但对机器而言反而是足够友好的，从而演变成了 Agent 时代的“天然工具”。

CLI的主要的特点如下：

- 零样本学习优势**：对于人类用户，记忆 grep、cat、vim 等 Linux / Unix命令及其参数是高门槛的；但对于大模型而言，这些命令是其预训练数据中海量代码和技术文档的一部分，属于“**先天知识**”。这意味着，让模型通过 CLI操作文件系统或网络，无需额外定义复杂的 API Schema（名称、描述、参数类型等），只需指令其使用标准命令即可。这节省了巨大的 token 空间和调试成本。

- 可扩展性与自解释性**：即使面对模型未曾见过的第三方 CLI 工具，只要遵循标准的 Linux/Unix 规范（如支持 --help），模型就能在运行时通过查询帮助文档，自主理解参数用法并执行调用。这种“按需查询、即时学习”的模式，完美契合了上下文工程的渐进式加载理念。

- Skill 集成**：新的第三方 CLI 工具可以通过 Skill 进行包装，在 Skill 的描述文件中提供安装指南和使用示例，使模型能够快速掌握新工具的使用。

同时，在 **Agent Skills** 体系中，**Resources** 形态的 **Script** 脚本逐渐成为工具承载的主流模式。无论是 Python 还是其他语言，具体的工具逻辑被封装为独立的脚本文件。这些脚本具有极强的灵活性：

1.本地与远程的统一：它们既可以直接执行本地命令（如文件操作、环境配置），也可以内部封装对远程 API 的调用。

2.协议的黑盒化：复杂的 API 鉴权、参数拼接等细节被隐藏在脚本内部，Agent 只需关注“调用哪个脚本”以及“传入什么核心参数”，极大地降低了模型理解的门槛。这也是为什么安装一个 Skill 往往就能赋予 Agent 处理复杂任务的能力——因为 Skill 不仅包含了 Prompt 指引，更内置了可执行的工具脚本。

综上所述，从早期的 Function Call 到如今的 CLI + Script 模式，Tools 的演进核心是从“人为适配模型”转向“利用模型原生能力”。我们不再试图为每一个操作编写专用的 API 接口，而是充分利用模型在预训练阶段积累的通用计算机操作知识（CLI）和代码执行能力（Script），构建更加轻量、灵活且易于扩展的工具生态。

Workflow：刚性编排 → 动态混合封装

第五个值得关注的演变维度，是 **Workflow（工作流）**。

在 Agent 发展的早期阶段，由于基础大模型的指令遵循能力和逻辑稳定性相对较弱，我们往往依赖显式的、硬编码的 Workflow 来保障任务的执行。这种模式类似于传统的“**状态机**”或“**流水线 Pipeline**”，将复杂任务拆解为严格固定的“第一步、第二步、第三步”，强制模型按部就班地执行。这种方式虽然牺牲了灵活性，但在当时是确保 Agent 不“跑偏”、不掉链子的必要手段。

然而，Workflow 也存在着很多的问题，比如运行过程非常“**机械化**”，无论 Agent 的外部环境发生了怎样的变化，Workflow 还仍然死板的严格遵循第一步、第二步...这样去运转，**无法根据实际情况做出动态调整**。但是，随着模型能力的不断跃升以及前文提到的 Agent Skills 体系的出

现，Workflow 的形态正在发生深刻的重构：从“刚性的流程编排”转向“动态的 Skill 封装与混合架构”。主要分为两部分：

- 逻辑内聚化**：原本分散在 Workflow 引擎中的步骤定义、约束条件、核心判断逻辑，现在可以直接写入 Skill 的 Markdown 描述文件（如 SKILL.md）中。模型通过阅读 Skills 的文档，即可理解任务的完整链路。

- 执行脚本化**：对于需要精确控制的环节，不再依赖外部工作流引擎的状态跳转，而是通过 Skill 关联的 Resources 的 Script 脚本进行代码级的编排和控制。这意味着，一个复杂的业务流程，现在可以被打包成一个独立的、可复用的 Skills。

这种转变带来了更大的灵活性和智能性，但也引入了新的挑战：可控性与稳定性的博弈。纯 Skill 驱动的模式赋予了 Agent 更高的自主性，但在面对极端复杂或容错率极低的场景时，模型仍可能出现理解偏差或执行跳跃，导致结果不可控。相比之下，传统的刚性 Workflow 虽然笨重，却提供了确定性的边界。

因此，当前的 Agent 研发范式正处于一个**新旧技术交叉融合的过渡期**。在企业级落地实践中，我们很少非此即彼地选择某一种方案，而是倾向于采用**混合架构**：

- 将成熟的、标准化的子任务封装为 **Skills**，通过Markdown文件来维护逻辑，利用其灵活性和易用性；

- Workflow 里的固定运行部分其实是可以全部转换为Script的，但是代码的可读性，很多时候没有Workflow方便。因此，将关键的、对稳定性要求极高的主干流程仍然保留为 **Workflow**，或者将特定的 Workflow 封装为一个特殊的 **Tool** 供 Agent 调用也是一个现阶段比较好的办法。

这种“Skill 为主，Workflow 为辅/兜底”的策略，既利用了新技术的红利，又保留了一定的确定性，是当前平衡开发效率与运行稳定性的最佳实践。

Environment：无状态 → 运行时环境

除了上述这些核心模块的演进，Agent 要想真正落地并稳定运行，还离不开一个至关重要的基础设施：**运行环境 (Environment / Runtime)**。

早期的 Agent 对工具调用、子Agent调用都是无状态的，几乎不需要所谓的“**运行环境**”。但是，随着 Agent 能力的增强，特别是引入了文件系统操作、代码执行等能力后，它不再仅仅是一个“问答机器”，而是一个需要持久化存储、文件读写和状态管理的“数字员工”。这就意味着，Agent 必须拥有一个专属的 **Workspace（工作空间）**。在这个工作空间中，Agent 可以安全地读取配置、写入日志、生成中间文件，甚至管理其 Skill 和 Memory 数据。

根据应用场景和安全要求的不同，这个 Runtime 环境主要呈现为两种形态：

- **本地个人电脑（Local Desktop）**：有着极高的便利性和灵活性。Agent 可以直接操作用户本地的文件系统、应用和网络，实现诸如“整理桌面文件”、“自动化办公流程”等贴近个人生活的复杂任务，OpenClaw 最早就是基于个人电脑的操作而“火”起来的。但是，由于缺乏严格的隔离机制，Agent 的操作失误可能导致用户重要数据丢失或系统配置混乱。因此，在本地环境中，通常需要引入更严格的用户确认机制或权限控制。

- **沙箱环境（Sandbox/Cloud Server）**：沙箱是企业级生产环境的主流选择。通过 Docker、Kubernetes 等容器化技术来构建隔离的沙箱，Agent 的所有操作被限制在特定的虚拟文件系统内。即使 Agent 执行了破坏性命令，也不会影响宿主机或其他服务。提供了必要的安全边界和资源管控，确保 Agent 在不可预测的行为下依然保持系统的整体稳定性。

03 总结

回顾全文，我们不难发现一个有趣的现象：从宏观架构上看，今天的 Agent 依然由 Prompt、Planning、Memory、Tools 等经典模块组成，这与 Lilian Weng 早期提出的理论框架并无二致。“**形**”未变，但“**神**”已大不同。

这是一场深刻的**内核重构**。在 Agent 的研发范式变化中，**Prompt** 从单体的“小作文”演变为解耦的上下文工程；**Planning** 从线性的 CoT 思维链升级为复杂的长程任务拆解；**Memory** 从传统的

前置向量检索转向文件系统化+向量检索的混合架构；**Tools** 从高成本的 API 封装回归到原生的 CLI 与脚本交互；**Workflow** 从刚性的外部编排内化为灵活的 Agent Skills 封装；**Environment** 从无状态的调用延伸为有状态的隔离运行时Runtime系统环境。

每一个模块背后的运行逻辑、数据流转方式以及工程实现范式，都发生了翻天覆地的变化。我们不再仅仅依赖模型的“智商”去硬扛所有问题，而是通过更精细的工程化手段（如文件化解耦、CLI 原生利用、沙箱隔离等）来弥补模型的不足，放大模型的优势。这也说明 Agent 正在从“魔法调优”到“系统工程”的转变，标志着 Agent 技术正在走向成熟，同时，更是我们对“**如何构建好的 Agent**”这一认知过程的不断深化。

虽然各个模块的具体实现方式仍在快速演进，但其核心目标始终未变：**在保证安全、可控的前提下，最大化释放模型的推理与执行潜力，让 Agent 真正成为能够解决复杂现实问题的得力助手。**

对于每一位从事 Agent 应用和落地的同学们而言，理解这些演进背后的逻辑，比掌握具体的某个工具更为重要。因为Agent还在持续发展，**模型会继续升级、工具会继续变化，框架会持续更新**，但这种“通过工程化手段构建确定性，以承载模型不确定性”的核心思想，将是未来很长一段时间内构建高质量 Agent 的基石。

作者说——

我也希望本文对 Agent 技术演进过程的梳理，能为大家在构建自己的 Agent 系统时，提供一些不一样的视角和思考。另外，我个人能力也有限，一定也有许多我没有考虑到或者没有总结出的一些方法论，也欢迎大家多多提出、交流，我们互相学习，共同提升，能够更好的利用 Agent 这个技术在自己所属的领域业务中应用落地。

本文是我个人技术探索的心路历程，一家之言、经验之谈，行文仓促，如有错误还请各位批评指正！

欢迎留言一起参与讨论~

修改于2026年6月1日

